Zephyr Project

IT-Schulungen.com
Technology & Transformation

Scheduling und Prioritäten

Auswahl, Präemption und Fairness

2.05

Was wir in 2.05 klären

Vom Ready-Set zur begründeten Ablaufvorhersage

01

Auswahl

höchste bereite Priorität

02

Klassen

kooperativ und präemptiv

03

Wechsel

Präemption, Blockieren und Yield

04

Fairness

Zeitscheiben und Starvation

05

Vorhersage

Scheduling-Zeitlinie erklären

Leitfrage: Welcher bereite Thread läuft als Nächstes – und warum?

Die Auswahlregel des Schedulers

ENTSCHEIDUNG

```
Ready Set → Klasse → Priorität → Gleichstand
```

1 · Ausführbar?	Nur ready Threads können ausgewählt werden
2 · Klasse	kooperative und präemptive Prioritäten unterscheiden
3 · Priorität	kleinere Zahl bedeutet innerhalb der Klasse höhere Priorität
4 · Gleichstand	FIFO-Verhalten oder konfigurierte Zeitscheibe beachten

Priorität bestimmt die Auswahlchance, nicht die Rechengeschwindigkeit eines Threads.

Drei Bereiche im Prioritätsmodell

01	`Kooperativ: Priorität < 0`	läuft bis Blockieren, Beenden oder freiwilliger Abgabe
02	`Präemptiv: Priorität ≥ 0`	höher priorisierter ready Thread kann übernehmen
03	`Gleiche Priorität`	optional Zeitscheiben für geeignete präemptive Threads

Merksatz: Erst die Klasse verstehen, dann die Zahlen innerhalb dieser Klasse vergleichen.

Drei Wege, die CPU abzugeben

PRÄEMPTION

höhere Priorität wird ready

laufender präemptiver Thread wird unterbrochen

Kontextwechsel erfolgt sofort

Ursache bleibt im Modell sichtbar

BLOCKIEREN

Thread wartet

Kerneloperation ist nicht sofort erfüllbar

k_sleep() wartet auf Zeit

ein anderer ready Thread läuft

YIELD

freiwillige Abgabe

`k_yield()` lässt Peers zu

Thread bleibt grundsätzlich ready

höhere Priorität gewinnt weiterhin

Für jede Ablaufänderung muss ein konkretes Ereignis benannt werden.

Scheduling-Zeitlinie lesen

SZENARIO

A: Prio 2 · B: Prio 1 · beide präemptiv

t0 · A läuft	B ist noch sleeping
t1 · B wird ready	B hat höhere Priorität als A
t1 · Präemption	A wechselt running → ready
t2 · B blockiert	A ist wieder der höchste ready Thread
t2 · A läuft weiter	Auswahl folgt Zustand und Priorität

Die Zeitlinie erklärt nicht nur die Reihenfolge, sondern auch jeden Zustandswechsel.

Blockieren oder freiwillig abgeben

BEISPIEL

```
void worker(void *a, void *b, void *c)
{
    while (1) {
        do_one_step();
        k_yield();
    }
}

/* Alternative nach einem Zyklus */
k_sleep(K_MSEC(10));
```

01 **k_yield()**

Thread bleibt ready und lässt Threads gleicher Priorität zum Zug kommen.
Kein Ersatz für ein echtes Warten auf ein Ereignis.

02 **k_sleep()**

Thread wechselt für die Wartezeit in sleeping.
Nach Ablauf wird er wieder ready und neu eingeordnet.

Jede blockierende Operation verändert das Ready Set und damit die nächste Auswahl.

Typische Scheduling-Fallen

Starvation	dauerhaft bereiter hochpriorer Thread verdrängt andere
Kooperative Endlosschleife	ohne Blockieren oder Yield bleibt die CPU gebunden
Priorität = Geschwindigkeit	falsch: Priorität beeinflusst Auswahl, nicht CPU-Takt
Zu viele Zeitannahmen	Reihenfolge ist nicht automatisch eine Deadline-Garantie
Prioritätsinversion	hohe Priorität wartet auf Ressource eines niedrigeren Threads

Vor jeder Änderung: Zustand, Priorität und auslösendes Ereignis gemeinsam prüfen.

Kurzcheck: Scheduling erklären

AUSWAHL

Schrittfolge

ready Threads bestimmen

Klasse einordnen

Prioritäten vergleichen

Gleichstand auflösen

WECHSEL

Ereignis benennen

höhere Priorität wird ready

Thread blockiert oder schläft

Thread beendet sich

Thread gibt freiwillig ab

PRAXIS 2.06

Vorhersage zuerst

```
Ausgabe beobachten
Abweichung markieren
Hypothese begründen
```

Transferfrage: Was müsste passieren, damit ein anderer Thread als vorhergesagt läuft?